NAME : VELMURUGAN

EXPERIMENT 41 TO 50

Queue Using Array

QUEUE USING ARRAYS

Write a C program to implement a Queue using Arrays with operations such as insertion (enqueue), deletion (dequeue), display, and peek. The program should simulate a ticket booking counter

```c
#include <stdio.h>

#define MAX 10

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x)
{
    if (rear == MAX - 1)
        printf("Queue Overflow\n");
    else
    {
        if (front == -1)
            front = 0;
        queue[++rear] = x;
    }
}

void dequeue()
```

Your INPUT goes here!

OUTPUT

```
Queue Elements: 101 102 103
Deleted Element: 101
Queue Elements: 102 103
```

Queue Using Linke

QUEUE USING LINKED LISTS

Write a C program to implement a Queue using Linked Lists for managing patients waiting in a hospital consultation room. The program should support enqueue, dequeue, display, and front operations

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int value)
{
    struct Node *newNode = malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL)
```

Your INPUT goes here!

```
    enqueue(201);
    enqueue(202);
    enqueue(203);

    display();

    dequeue();

    printf("Remaining Queue: ");
    display();

    return 0;
}
```

OUTPUT

```
Queue Elements: 201 202 203
Deleted Patient ID: 201
Remaining Queue: Queue Elements: 202 203
```

Double Ended Que

DOUBLE ENDED QUEUE

Write a C program to implement a Double Ended Queue (Deque) using Arrays where insertion and deletion can be performed from both front and rear ends. Consider a warehouse inventory management system

```c
#include <stdio.h>

#define MAX 10

int deque[MAX];
int front = -1, rear = -1;

void insertFront(int x)
{
    if (front == -1)
        front = rear = 0;
    else
        front--;

    deque[front] = x;
}

void insertRear(int x)
{
    if (rear == -1)
```

```c
    printf("\n");
}

int main()
{
    insertFront(100);
    insertRear(200);
    insertFront(50);

    deleteRear();
    display();

    return 0;
}
```

OUTPUT

```
Queue Elements: 50 100 200
```

Priority Queue Usir

PRIORITY QUEUE USING ARRAYS

Write a C program to implement a Priority Queue using Arrays for processing emergency service requests. Requests with higher priority must be processed before lower-priority requests irrespective of arrival

```c
#include <stdio.h>

#define MAX 10

int queue[MAX];
int priority[MAX];
int n = 0;

void insert(int value, int p)
{
    queue[n] = value;
    priority[n] = p;
    n++;
}

void delete()
{
    int i, pos = 0;
```

```
int main()
{
    insert(101, 3);
    insert(102, 1);
    insert(103, 2);

    delete();
    display();

    return 0;
}
```

OUTPUT

```
Processed Request: 102
Remaining Queue: 101 103
```

Queue Data Structu

QUEUE DATA STRUCTURE

Write a C program to simulate a Railway Reservation Waiting List using Queue data structure. Passengers enter the waiting queue and are allocated seats as cancellations occur. The program should

```
#include <stdio.h>

int queue[10];
int front = 0, rear = -1;

void addPassenger(int id)
{
    queue[++rear] = id;
}

void cancel()
{
    if (front <= rear)
    {
        printf("Passenger %d Confirmed\n", queue[front]);
        front++;
    }
}

void display()
```

```
    printf("\n");
}

int main()
{
    addPassenger(501);
    addPassenger(502);

    cancel();
    display();

    return 0;
}
```

OUTPUT

```
Passenger 501 Confirmed
Waiting List: 502
```

Queue

QUEUE

Write a C program to implement a Queue for managing online food delivery orders where orders are received and processed in the order they arrive. The program should allow adding new orders.

```c
#include <stdio.h>

int queue[10];
int front = 0, rear = -1;

void addOrder(int order)
{
    queue[++rear] = order;
}

void processOrder()
{
    if (front <= rear)
        printf("Processed Order: %d\n", queue[front++]);
}

void display()
{
    int i;
```

```
    printf("\n");
}

int main()
{
    addOrder(1001);
    addOrder(1002);
    addOrder(1003);

    processOrder();
    display();

    return 0;
}
```

OUTPUT

```
Processed Order: 1001
Pending Orders: 1002 1003
```

Queue Using Linke

QUEUE USING LINKED LISTS -2

Write a C program to implement a Queue using Linked Lists for managing network packets in a router. Packets should be transmitted in the order received, and the program must support dynamic

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int value)
{
    struct Node *newNode = malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL)
```

```
    printf("\n");
}

int main()
{
    enqueue(11);
    enqueue(12);
    enqueue(13);

    dequeue();
    display();

    return 0;
}
```

OUTPUT

```
Transmitted Packet: 11
Remaining Packets: 12 13
```

Circular Queue

CIRCULAR QUEUE

Write a C program to implement a Circular Queue for a traffic signal management system where vehicles are processed lane by lane in a cyclic manner. The system should continuously rotate through

```c
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int value)
{
    if ((rear + 1) % MAX == front)
        return;

    if (front == -1)
        front = 0;

    rear = (rear + 1) % MAX;
    queue[rear] = value;
}

void dequeue()
```

```
    printf("\n");
}

int main()
{
    enqueue(21);
    enqueue(22);
    enqueue(23);

    dequeue();
    display();

    return 0;
}
```

OUTPUT

```
Vehicle Passed: 21
Queue: 22 23
```

Queue-2

QUEUE-2

Write a C program to implement a Queue for managing examination answer sheet evaluation. Answer sheets submitted by students are evaluated in the same sequence in which they are

```c
#include <stdio.h>

int queue[10];
int front = 0, rear = -1;

void enqueue(int id)
{
    queue[++rear] = id;
}

void dequeue()
{
    if (front <= rear)
        printf("Evaluated Sheet: %d\n", queue[front++]);
}

void display()
{
    int i;

```

```c

int main()
{
    enqueue(401);
    enqueue(402);
    enqueue(403);

    dequeue();
    display();

    return 0;
}
```

OUTPUT

```
Evaluated Sheet: 401
Pending Sheets: 402 403
```

Queue Using Linke

QUEUE USING LINKED LIST-3

Write a C program to simulate a supermarket billing queue using Linked List implementation. Customers join the billing queue and are billed in FIFO order. The queue size should dynamically increase

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int value)
{
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;
```

```
    printf("\n");
}

int main()
{
    enqueue(11);
    enqueue(12);
    enqueue(13);

    dequeue();
    display();

    return 0;
}
```

OUTPUT

```
Billed Customer: 11
Remaining Customers: 12 13
```